NumPy Kurvendiskussion & Ableitungen

Mathematische Funktionen mit Python visualisieren

Studienkolleg · Rahn Education

Einführung in NumPy

Was ist NumPy?

NumPy – Numerical Python

Grundlegende Bibliothek für wissenschaftliches Rechnen in Python.

- Leistungsstarke Funktionen für Arrays und Matrizen
- Mathematische Operationen auf ganzen Arrays
- Zusammen mit Matplotlib ideal für Kurvendiskussion

Bibliotheken importieren

```python
import numpy as np
import matplotlib.pyplot as plt
```

Konvention

`np` und `plt` sind die Standard-Abkürzungen – immer so verwenden.

Funktionen definieren und visualisieren

Funktion definieren

Beispiel: $f(x) = x^2 - 4x + 3$

```
def f(x):
    return x**2 - 4*x + 3
```

Funktion definieren

Beispiel: $f(x) = x^2 - 4x + 3$

```python
def f(x):
    return x**2 - 4*x + 3
```

```python
f(10)    # Ausgabe: 63
```

Wenn `x` ein NumPy-Array ist, wird `f` automatisch auf jeden Wert angewendet.

Funktion visualisieren

np.linspace() + plt.plot()

```
x = np.linspace(-10, 10, 100)  # 100 Punkte von -10 bis 10
y = f(x)                       # f auf alle x-Werte anwenden

plt.figure(figsize=(10, 6))
plt.plot(x, y, 'b-', linewidth=2)
plt.grid(True, alpha=0.3)
plt.title('f(x) = x^2 - 4x + 3')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

np.linspace() erklärt

np.linspace(start, stop, num)

Erzeugt ein Array gleichmäßig verteilter Werte.

- `start` – erster Wert
- `stop` – letzter Wert
- `num` – Anzahl der Punkte (mehr = glattere Kurve)

Mehrere Funktionen vergleichen

Mehrere Funktionen im selben Plot

```
def g(x):
    return 2 * np.sin(2 * x)

def h(x):
    return np.sin(x)

x = np.linspace(-2*np.pi, 2*np.pi, 1000)

plt.plot(x, g(x), 'b-', label='2*sin(2x)')
plt.plot(x, h(x), 'r-', label='sin(x)')
plt.legend()
plt.show()
```

NumPy-Funktionen für Trigonometrie

- `np.sin(x)` – Sinus
- `np.cos(x)` – Kosinus
- `np.exp(x)` – Exponentialfunktion e^x
- `np.log(x)` – Natürlicher Logarithmus
- `np.pi` – Die Zahl π

Numerische Ableitungen

Das Konzept der Ableitung

Definition der Ableitung

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Die Ableitung gibt die **Steigung der Tangente** an der Kurve an einem Punkt an.

Numerisch: wir wählen ein sehr kleines h (z.B. $h = 0{,}01$).

Ableitung implementieren

```python
def ableitung(func, x, h=0.01):
    """Numerische Ableitung einer Funktion an Stelle x."""
    return (func(x + h) - func(x)) / h
```

Ableitung implementieren

```python
def ableitung(func, x, h=0.01):
    """Numerische Ableitung einer Funktion an Stelle x."""
    return (func(x + h) - func(x)) / h
```

```python
def f(x):
    return x**2    # f'(x) = 2x

print(ableitung(f, 1))    # ~2.01
print(ableitung(f, 2))    # ~4.01
print(ableitung(f, 5))    # ~10.01
```

Numerische vs. analytische Ableitung

Kleine Ungenauigkeit ist normal

Die numerische Ableitung ist eine **Approximation**.

Kleineres $h \to$ genaueres Ergebnis, aber zu klein kann zu Rundungsfehlern führen.

Für die meisten Anwendungen ist $h = 0{,}01$ ausreichend.

Funktion und Ableitung visualisieren

Funktion und Ableitung definieren

```python
def f(x):
    return x**3 - 6*x**2 + 9*x - 3

def df(x, h=0.01):
    return (f(x + h) - f(x)) / h

x  = np.linspace(-1, 5, 1000)
y  = f(x)
dy = df(x)
```

Zweigeteilter Plot

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 1, 1)            # oberes Teilbild
plt.plot(x, y,  'b-', linewidth=2, label="f(x)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.title("Funktion f(x)")

plt.subplot(2, 1, 2)            # unteres Teilbild
plt.plot(x, dy, 'r-', linewidth=2, label="f'(x)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.title("Ableitung f'(x)")

plt.tight_layout()
plt.show()
```

Übungsaufgaben

Aufgabe 1 & 2

Aufgabe 1 – Grundlegende Funktionsdarstellung

Erstelle $f(x) = x^2 - 2x - 3$ und visualisiere sie für $x \in [-3, 5]$. Markiere die Nullstellen im Diagramm.

Aufgabe 2 – Mehrere Funktionen vergleichen

Visualisiere $f(x) = x^2$, $g(x) = 2x^2 - 4$ und $h(x) = 0{,}5x^2 + 2$ in einem gemeinsamen Koordinatensystem mit Legende.

Aufgabe 3 & 4

Aufgabe 3 – Numerische Ableitung

Implementiere $f(x) = x^3 - 3x^2 + 2$ und berechne die numerische Ableitung an $x = 0$, $x = 1$, $x = 2$.
Vergleiche mit den analytischen Werten.

Aufgabe 4 – Visualisierung

Stelle $f(x) = \sin(x) \cdot \cos(x)$ und ihre Ableitung in einem zweigeteilten Diagramm dar.
Bereich: $x \in [-2\pi, 2\pi]$.

Aufgabe 5 & 6

Aufgabe 5 – Extremwerte finden

Schreibe eine Funktion, die lokale Extremwerte durch Nullstellen der ersten Ableitung findet.
Teste mit $f(x) = x^3 - 6x^2 + 9x + 1$ und visualisiere die Extrempunkte.

Aufgabe 6 – Wendepunkte bestimmen

Implementiere die zweite Ableitung. Finde die Wendepunkte von $f(x) = x^4 - 4x^3 + 6x^2$ durch Nullstellen der zweiten Ableitung und visualisiere sie.

Fragen?